FI Agent

Field Investigation Credit Verification System

Technical Architecture & System Documentation

Version 2.0 · May 2026

■ **Android App**	Camera, Microphone, GPS, WebSocket client
■ **FastAPI Server**	WebSocket conductor, REST upload, PDF reports
■ **AWS Polly**	Neural Text-to-Speech (Indian English — Kajal)
■ **AWS Transcribe**	Real-time STT streaming (en-IN)
■ **OpenAI GPT-4o**	Property image analysis for credit assessment
■ **ReportLab PDF**	Auto-generated credit verification reports

CONFIDENTIAL — FOR INTERNAL USE ONLY

Table of Contents

1. System Overview

1.1 Purpose

FI Agent is a **Field Investigation Credit Verification** system for small-loan disbursement teams. A field officer visits the applicant's residence, launches the Android app, and conducts a server-orchestrated interview: structured questions are spoken aloud (AWS Polly), the applicant's answers are recorded and transcribed in real time (AWS Transcribe), and property photographs are captured. At session end the server automatically runs geo-verification, property analysis (OpenAI GPT-4o), and produces a formatted PDF credit report — all without manual data entry.

1.2 High-Level Architecture

Client	Android App	Kotlin / CameraX / OkHttp WS	UI, camera, mic, GPS, WS client
Comms	WebSocket	FastAPI / OkHttp3	Bidirectional real-time session
Server	Session Conductor	Python asyncio	Orchestrates entire FI flow
AI/TTS	AWS Polly	boto3 Neural en-IN (Kajal)	Text → MP3 audio
AI/STT	AWS Transcribe	amazon-transcribe SDK streaming	PCM audio → live transcript
AI/Vision	OpenAI GPT-4o	openai Python SDK	Property image analysis
Storage	Session Files	Filesystem D:\fig\	Photos, recording, metadata JSON
Output	PDF Report	ReportLab	server/reports/ credit report

1.3 Key Capabilities

- Server-driven protocol — Android is a thin terminal; all session logic lives on the server
- Real-time Indian-English voice questions via AWS Polly (Neural Kajal voice)
- Live STT transcript displayed on screen as the applicant speaks (AWS Transcribe en-IN)
- GPS captured at every Q&A; and photo event, verified against configurable 500 m radius
- Session audio recorded in full (foreground service) for audit trail
- OpenAI GPT-4o analyses each property photo: condition, credit indicators, score 1–10
- Auto-generated A4 PDF report with embedded photo thumbnails and geo-verification table
- All questions and photo prompts configurable via environment variables — no code change

2. Android Application

2.1 Package Structure & Layers

The app follows a strict layered architecture. Each layer has a single responsibility and depends only on the layer below it.

com.ficx.app	Root	MainActivity — Start FI button, permission gating
com.ficx.app.config	Config	FiConfig — all tunable parameters
com.ficx.app.domain.model	Domain	FiSession, SessionStep, QuestionAnswer, PhotoCapture, GeoPoint
com.ficx.app.domain.usecase	Domain	BuildSessionStepsUseCase — builds ordered step list
com.ficx.app.data.model	Data	DTO classes (Gson-serialisable) for REST upload
com.ficx.app.data.network	Data	FiApiService (Retrofit), FiApiClient (OkHttp)
com.ficx.app.data.repository	Data	FiSessionRepository — multipart upload
com.ficx.app.service	Service	WebSocketManager, AudioRecorder, AudioPlayer, LocationHelper, SessionRecordingService, FiLog
com.ficx.app.ui.viewmodel	UI	FiSessionViewModel — WS message handler, state machine
com.ficx.app.ui	UI	FiSessionActivity — camera, layout, lifecycle

2.2 Permissions

CAMERA	CameraX preview, photo capture (front + back)
RECORD_AUDIO	AudioRecord PCM stream → server STT + session recording
ACCESS_FINE_LOCATION	FusedLocationProvider — lat/long per Q&A; and photo
INTERNET	WebSocket + REST upload to server
READ_MEDIA_IMAGES / READ_EXTERNAL_STORAGE	Access saved session files (API-version gated)
FOREGROUND_SERVICE (camera|microphone)	SessionRecordingService — keeps recording across lifecycle
WAKE_LOCK	Prevent screen-off during session

2.3 Session Flow (Android side)

- 1. User taps Start FI → EasyPermissions requests all required permissions
- 2. FiSessionActivity starts: front camera binds, SessionRecordingService starts foreground service
- 3. FiSessionViewModel connects WebSocket to ws:///ws/fi-session/
- 4. Sends {type:"ready", session_id, device_id, started_at} handshake
- 5. Handles incoming server messages and updates UiState flow (observed by Activity)
- 6. On tts_audio → AudioPlayer decodes base64 MP3, plays via MediaPlayer, sends tts_done
- 7. On start_listening → AudioRecorder sends 100 ms PCM chunks as binary WS frames
- 8. On capture_photo → CameraX takePicture(), base64 JPEG sent as {type:"photo"} JSON
- 9. On session_done → stop recording service, toast message, Activity finishes after 3 s

2.4 Local Storage (Android)

Session files	getFilesDir()/fi_sessions//
Photos	photo_.jpg
Session recording	recording__.aac
TTS temp files	getCacheDir()/tts_.mp3 (deleted after playback)

2.5 FiConfig Reference (android/.../config/FiConfig.kt)

serverUrl	http://192.168.1.100:8000	FastAPI server base URL
questions	3 defaults	Spoken Q&A; questions (configurable list)
selfPhotoPrompt	See code	Text spoken before the selfie countdown
photoPrompts	4 rooms	Text + countdown for each room photo
countdownSeconds	10	Countdown timer before each photo
sttTimeoutMs	10 000	Max recording duration per question (ms)
uploadTimeoutSec	120	HTTP upload timeout for REST batch upload

3. Server Application

3.1 Directory Structure

```
server/
■■■ main.py                    FastAPI app, logging setup, router registration
■■■ config.py                  Pydantic-settings; all env-var config + property helpers
■■■ requirements.txt           Python dependencies
■■■ .env                       (not committed) secrets and overrides
■■■ models/
■   ■■■ fi_session.py          Pydantic models: SessionMetadata, PhotoMeta, GeoPoint …
■■■ routers/
■   ■■■ fi_session.py          POST /api/fi-session/upload  (REST batch upload)
■   ■■■ ws_session.py          WS  /ws/fi-session/{session_id}
■■■ services/
    ■■■ aws_service.py         PollyService + TranscribeService + singletons
    ■■■ geo_service.py         Haversine, centroid, radius verification
    ■■■ openai_service.py      GPT-4o batch image analysis
    ■■■ report_service.py      ReportLab A4 PDF generation
    ■■■ session_conductor.py   WebSocket orchestration + background report trigger
    ■■■ storage_service.py     Async file I/O into D:\fig\\

docs/
■■■ generate_docs.py           This script
■■■ fi_agent_documentation.pdf

reports/
■■■ /
    ■■■ fi_report__.pdf
```

3.2 API Endpoints

WebSocket	/ws/fi-session/{session_id}	Real-time FI session — server-driven conductor
POST (multipart)	/api/fi-session/upload	Legacy/batch upload after offline session
GET	/api/fi-session/{id}/files	List files stored for a session
GET	/health	Server status + active config summary

3.3 Service Layer Responsibilities

session_conductor	WebSocket message loop; calls all other services in sequence; triggers report generation as background asyncio task
aws_service	PollyService: boto3 synthesize_speech → base64 MP3. TranscribeService: async streaming PCM queue → final transcript
geo_service	Collects GeoPoints from SessionMetadata; Haversine centroid check; returns per-point distance table and verified boolean
openai_service	Encodes each photo as base64; sends to GPT-4o with credit-officer system prompt; runs all images concurrently with asyncio.gather

report_service	ReportLab A4 PDF: cover, Q&A; table, geo table, photo+analysis panels, credit summary; saved to server/reports//
storage_service	Async aiofiles writes to D:\fig\\; creates directories on first use

3.4 Server Storage Layout

Session root	D:\fig\ (FI_STORAGE_ROOT env var)
Session folder	D:\fig\\
Photos	D:\fig\\photo_.jpg
Recording	D:\fig\\recording__.aac
Metadata	D:\fig\\metadata.json
Reports root	server/reports\ (relative to server working dir)
Report PDF	server/reports\\fi_report__.pdf

4. WebSocket Communication Protocol

4.1 Connection & Handshake

URL	ws://:/ws/fi-session/
Transport	OkHttp3 WebSocket (Android) ↔ FastAPI WebSocket (server)
Text frames	UTF-8 JSON — all control messages
Binary frames	Raw PCM audio chunks from Android during listening phases
Handshake	Android sends {type:"ready"} after connection opens; server begins session after receiving it

4.2 Message Reference Table

Direction: S→A = Server to Android, A→S = Android to Server, BIN = binary frame.

ready	A→S	session_id, device_id, started_at	Handshake — starts the session
question	S→A	index, text	Display question text on screen
tts_audio	S→A	format='mp3', data=	Polly MP3 to play on device
tts_done	A→S	—	Playback complete; server may proceed
start_listening	S→A	question_index, timeout_ms	Android starts AudioRecord and streams PCM
	A→S	binary frames	Raw 16 kHz / 16-bit / mono PCM
audio_end	A→S	—	Recording stopped; Transcribe stream closed
transcript	S→A	question_index, text, is_final	Partial (live) or final answer text
announce_photo	S→A	prompt, is_selfie	Show prompt; play TTS
countdown	S→A	value (10 … 1)	Countdown tick — shown as large number
capture_photo	S→A	prompt, is_selfie, photo_index	CameraX takePicture() triggered
photo	A→S	filename, data=, prompt, photo_index, geo={lat,lon,ts}	Captured photo + GPS
photo_ack	S→A	photo_index	Server saved photo successfully
session_done	S→A	session_id, message	All steps complete; report generating
error	S→A	message	Fatal error — session terminated

4.3 Audio Streaming Format

Encoding	PCM signed 16-bit little-endian
Sample rate	16 000 Hz
Channels	Mono (1 channel)
Chunk size	~3 200 bytes per frame (100 ms of audio)
Frame type	WebSocket binary frame

End signal	JSON text frame {type: 'audio_end'}
AWS requirement	Transcribe streaming expects exactly this format for en-IN

5. End-to-End Sequence Diagrams

5.1 Complete FI Session Flow

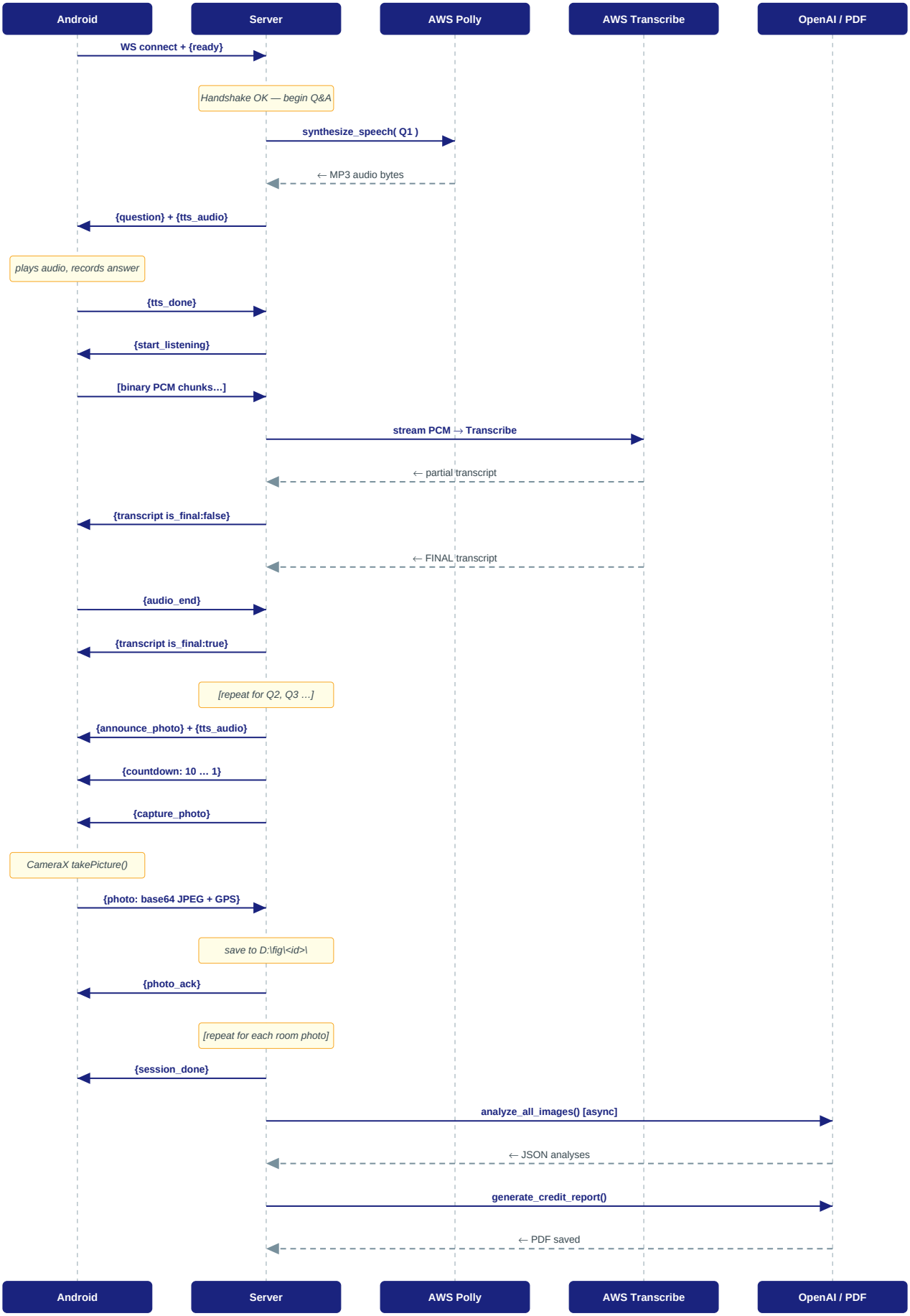

Figure 5.1 — Complete WebSocket session with Q&A;, photos, and background report generation

5.2 Q&A; Detail — AWS Polly TTS + AWS Transcribe STT

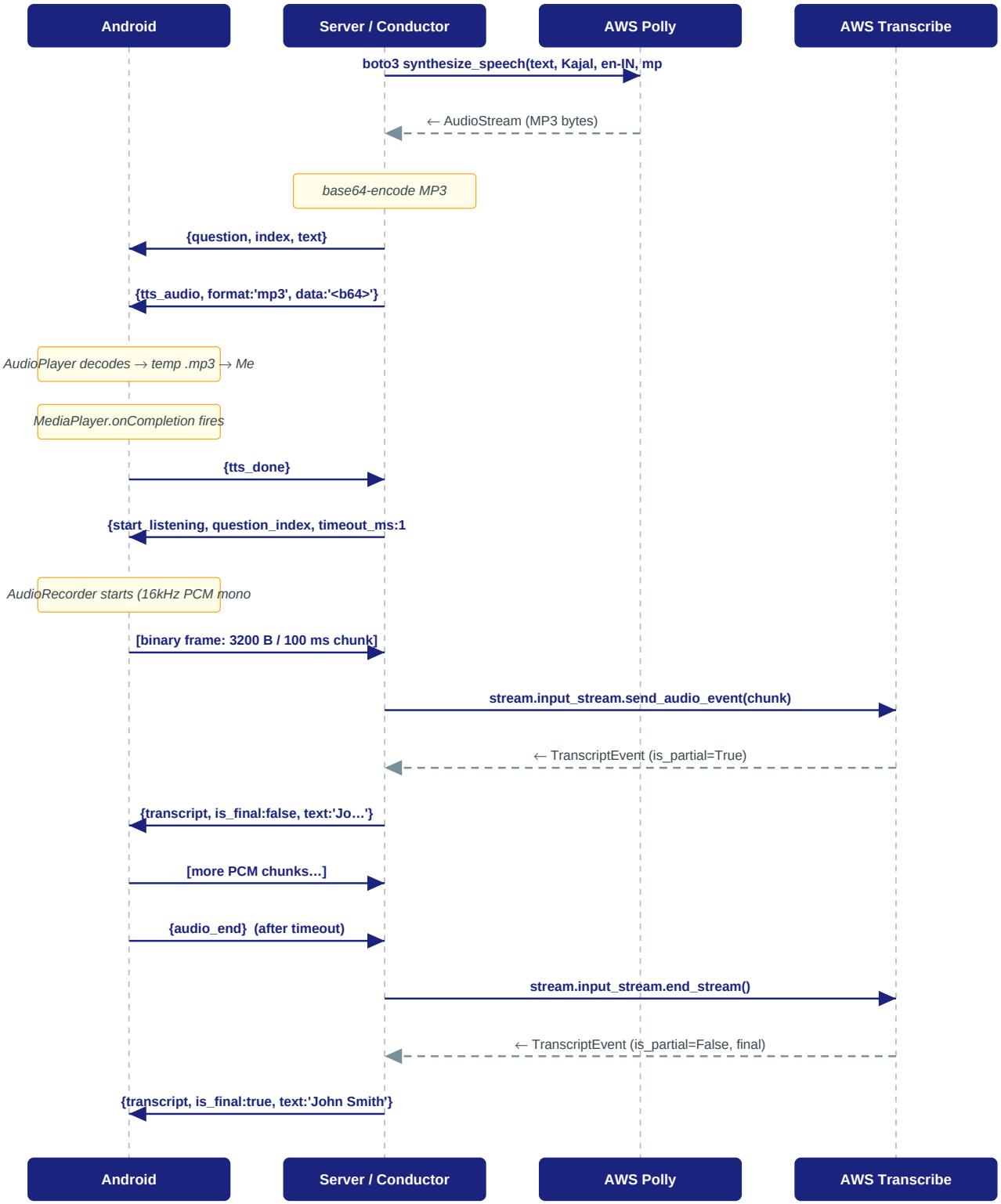

Figure 5.2 — Single question: Polly TTS playback followed by Transcribe STT

5.3 Photo Capture & Upload

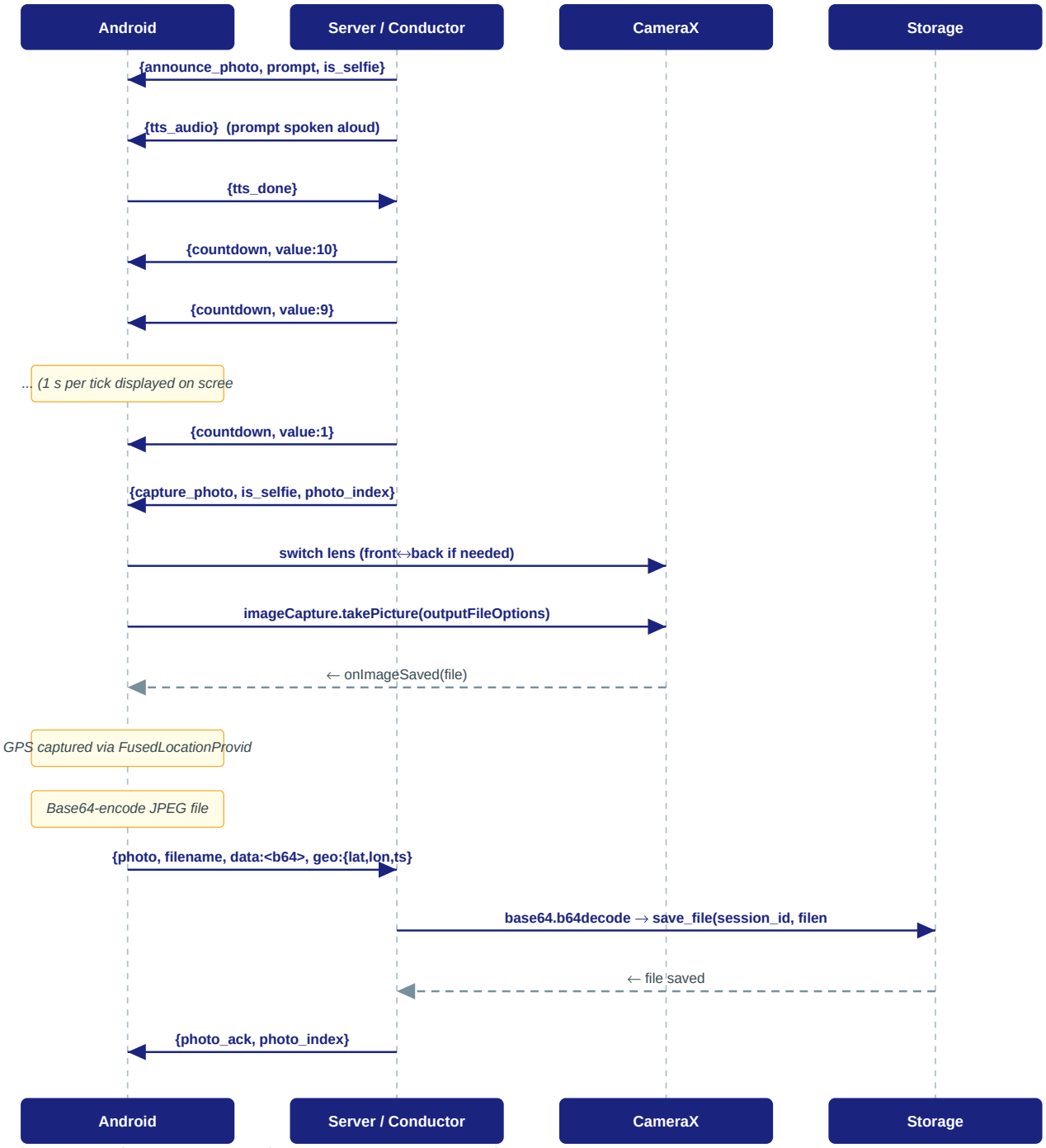

Figure 5.3 — Photo capture: countdown → CameraX → GPS tagging → server save

5.4 Post-Session Report Generation Pipeline

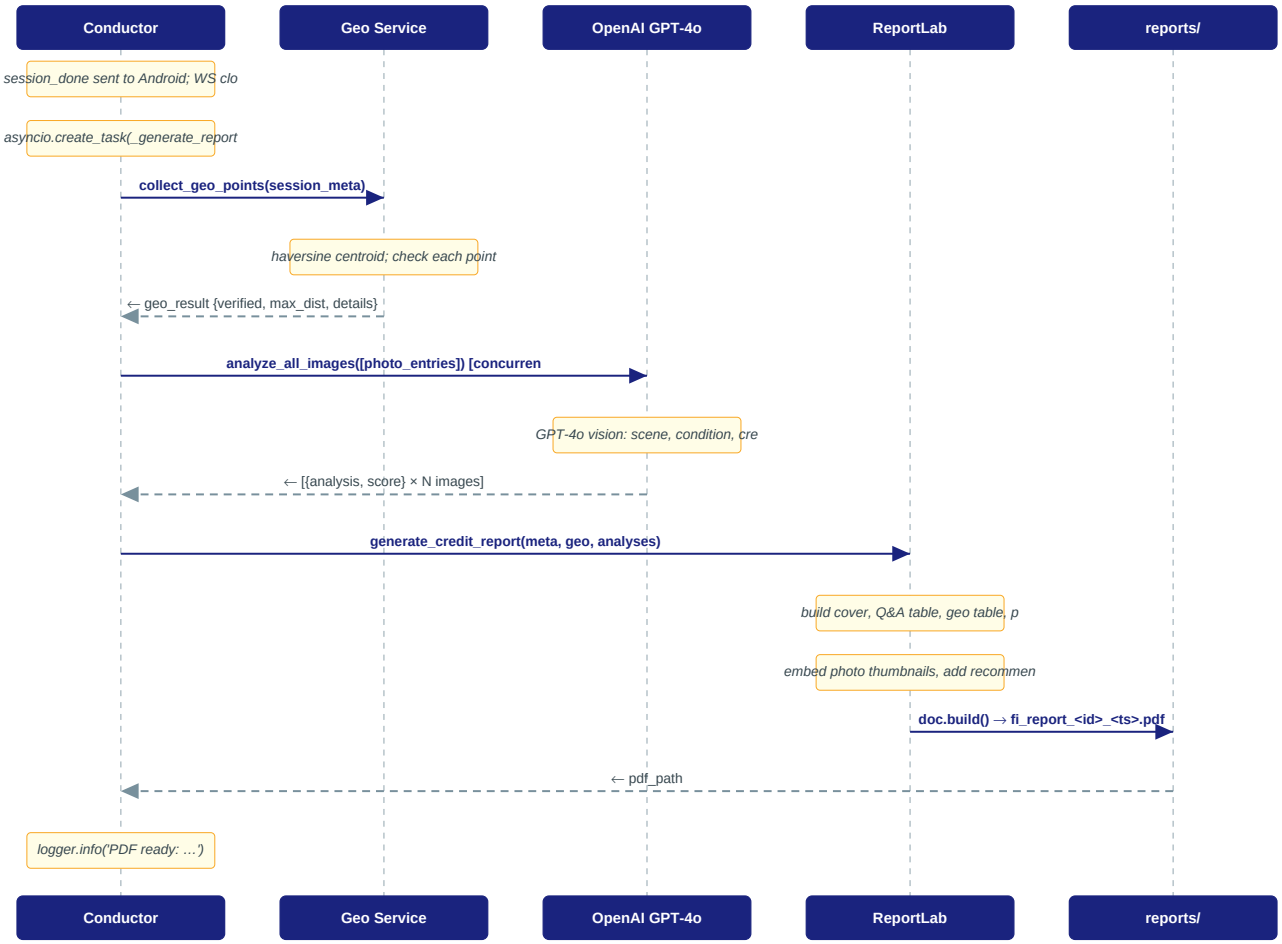

Figure 5.4 — Background report pipeline: geo verification, GPT-4o analysis, PDF build

6. AWS Services Integration

6.1 AWS Polly — Text-to-Speech

Voice ID	Kajal (Neural, Female, Indian English)
Language Code	en-IN
Engine	neural
Output Format	mp3
Invocation	boto3 client: polly.synthesize_speech(…)
Called by	PollyService.synthesize() → async_synthesize() in services/aws_service.py
Fallback	If Polly fails, question is sent as text only; TTS skipped gracefully

6.2 AWS Transcribe Streaming — Speech-to-Text

Language Code	en-IN (Indian English)
Media Encoding	pcm
Sample Rate	16 000 Hz
SDK Package	amazon-transcribe (pip install amazon-transcribe)
Invocation	client.start_stream_transcription(…) then send_audio_event per chunk
Called by	TranscribeService.transcribe_stream() in services/aws_service.py
Partial results	Sent to Android in real time as {type:'transcript', is_final:false}
Final result	Assembled from all non-partial TranscriptEvent segments

6.3 Required IAM Permissions

Create an IAM user or role with the following minimum policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "polly:SynthesizeSpeech",
        "transcribe:StartStreamTranscription"
      ],
      "Resource": "*"
    }
  ]
}
```

Credentials are provided via environment variables FI_AWS_ACCESS_KEY_ID and FI_AWS_SECRET_ACCESS_KEY, or via an EC2 instance role / ECS task role (recommended for production).

7. Storage Architecture

7.1 Android Local Storage

```
/data/data/com.ficx.app/files/           ← getFilesDir()
■■■ fi_sessions/
    ■■■ session_/
        ■■■ photo_20260503_102345_001.jpg  ← front camera (selfie)
        ■■■ photo_20260503_102410_002.jpg  ← hall
        ■■■ photo_20260503_102445_003.jpg  ← kitchen
        ■■■ photo_20260503_102520_004.jpg  ← bedroom
        ■■■ photo_20260503_102555_005.jpg  ← outside
        ■■■ recording_session__.aac  ← full session audio

/data/data/com.ficx.app/cache/           ← getCacheDir()
■■■ tts_.mp3                  ← temporary, deleted after playback
```

7.2 Server Session Storage

```
D:\fig\                                  ← FI_STORAGE_ROOT (configurable)
■■■ session_\
    ■■■ photo_20260503_102345_001.jpg     ← received via WebSocket binary
    ■■■ photo_20260503_102410_002.jpg
    ■■■ photo_20260503_102445_003.jpg
    ■■■ photo_20260503_102520_004.jpg
    ■■■ photo_20260503_102555_005.jpg
    ■■■ recording_session__.aac  ← uploaded via REST (optional)
    ■■■ metadata.json                    ← session Q&A;, photo meta, timestamps
```

metadata.json schema:

```
{
  "session_id": "session_1746291234567",
  "device_id":  "a1b2c3d4e5f6",
  "started_at": "2026-05-03T10:20:00Z",
  "ended_at":   "2026-05-03T10:28:45Z",
  "questions": [
    { "question": "What is your name?",
      "answer":   "Ramesh Kumar",
      "geo":      { "latitude": 19.0760, "longitude": 72.8777, "timestamp": "…" } }
  ],
  "photos": [
    { "prompt": "I am taking your photo now…",
      "filename": "photo_20260503_102345_001.jpg",
      "geo": { "latitude": 19.0761, "longitude": 72.8778, "timestamp": "…" } }
  ],
  "recording_filename": null
}
```

7.3 Reports Directory

```
server/                                  ← working directory when uvicorn runs
■■■ reports/                             ← created automatically on first use
    ■■■ session_/
        ■■■ fi_report_session__20260503_102900.pdf
```

7.4 File Naming Conventions

Photo (Android)	photo_.jpg	SimpleDateFormat timestamp
Photo (Server)	Same as received from Android	Preserved as-is
Session recording	recording__.aac	AAC 128 kbps, 44.1 kHz
Session metadata	metadata.json	One per session folder
Credit report PDF	fi_report__.pdf	UTC timestamp

8. Report Generation Pipeline

8.1 Geo-Verification Logic

After the session ends the server collects every GPS coordinate captured during the session — one per Q&A; answer and one per photo. All points must lie within the configured radius (default 500 m) of the geographic centroid to be considered co-located.

```
Algorithm (services/geo_service.py)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
1. collect_geo_points(meta)
   Yields all GeoPoint objects from meta.questions[].geo
                     and from meta.photos[].geo

2. centroid = mean(latitudes), mean(longitudes)

3. For each point P:
   d = haversine(centroid, P)           # in metres
   if d > geo_radius_meters → OUTLIER

4. verified = (outliers == [])

Haversine formula:
  R = 6 371 000 m
  Δϕ = lat2 - lat1  (radians)
  Δλ = lon2 - lon1  (radians)
  a = sin²(Δϕ/2) + cos(ϕ1)·cos(ϕ2)·sin²(Δλ/2)
  d = R · 2 · atan2(√a, √(1−a))
```

8.2 OpenAI Image Analysis

Model	gpt-4o (configurable via FI_OPENAI_MODEL)
Detail level	low (faster, lower cost; sufficient for property assessment)
Concurrency	asyncio.gather — all photos analysed in parallel
System prompt	Experienced FI officer conducting home-visit credit verification
User prompt	Structured template requesting: Scene Type, Condition, Key Observations, Credit Indicators (positive / concerns), Assessment Score 1–10
Max tokens	450 per image
Fallback	Analysis text set to '■ Analysis skipped' if key not configured or API fails

8.3 PDF Report Structure (services/report_service.py)

Cover header	FI Agent logo area, session ID, device ID, start/end times, report date
1. Customer Details	Q&A; answers displayed as key-value pairs; keywords matched to Full Name, DOB, Address
2. Location Verification	Status badge (VERIFIED / FAILED), centroid co-ordinates, per-point table with distance and colour-coded pass/fail
3. Image Analysis	One panel per photo: thumbnail (6.5 cm wide) beside GPT-4o analysis text; condition, observations, credit indicators, score

4. Credit Assessment	Summary table: customer info, geo status, photo count, AI model used, Recommendation (RECOMMEND / FURTHER VERIFICATION REQUIRED)
Footer	Page number, generated timestamp, confidentiality notice

9. Configuration Reference

9.1 Server — .env Variables

All variables are prefixed FI_ and read by pydantic-settings at startup.

FI_STORAGE_ROOT	D:\\fig	Root folder for session files
FI_HOST	0.0.0.0	Uvicorn bind host
FI_PORT	8000	Uvicorn bind port
FI_AWS_REGION	us-east-1	AWS region for Polly + Transcribe
FI_AWS_ACCESS_KEY_ID	(empty)	AWS credentials (or use instance role)
FI_AWS_SECRET_ACCESS_KEY	(empty)	AWS credentials
FI_POLLY_VOICE_ID	Kajal	Polly voice (Kajal = Neural en-IN)
FI_TRANSCRIBE_LANGUAGE_CODE	en-IN	Transcribe language code
FI_OPENAI_API_KEY	(empty)	OpenAI API key for image analysis
FI_OPENAI_MODEL	gpt-4o	OpenAI model (gpt-4o or gpt-4o-mini)
FI_GEO_RADIUS_METERS	500	Max distance from centroid for geo-verify
FI_COUNTDOWN_SECONDS	10	Seconds of countdown before each photo
FI_LISTEN_TIMEOUT_MS	12000	Max STT recording duration per question
FI_SELF_PHOTO_PROMPT	See config.py	TTS text before selfie countdown
FI_FI_QUESTIONS_JSON	3 defaults	JSON array of question strings
FI_FI_PHOTO_PROMPTS_JSON	4 room defaults	JSON array of photo prompt strings

Example .env file:

```
FI_STORAGE_ROOT=D:\fig
FI_AWS_REGION=ap-south-1
FI_AWS_ACCESS_KEY_ID=AKIA...
FI_AWS_SECRET_ACCESS_KEY=wJalrX...
FI_OPENAI_API_KEY=sk-proj-...
FI_GEO_RADIUS_METERS=300
FI_POLLY_VOICE_ID=Kajal
FI_TRANSCRIBE_LANGUAGE_CODE=en-IN
FI_FI_QUESTIONS_JSON=["What is your name?","What is your date of birth?","What is your address?"]
```

9.2 Android — FiConfig (config/FiConfig.kt)

Edit FiConfig.kt to change Android-side settings. The server URL must point to the machine running the FastAPI server.

```
data class FiConfig(
    val serverUrl          : String = "http://192.168.1.100:8000",
    val questions          : List = listOf(…),  // overridden by server
    val selfPhotoPrompt    : String = "…",              // for local reference
    val photoPrompts       : List = listOf(…),  // for local reference
    val countdownSeconds   : Int    = 10,
```

```
    val sttTimeoutMs       : Long   = 10_000L,
    val uploadTimeoutSec   : Long   = 120L
)
```

Note: questions and photo prompts are actually driven by the server. The FiConfig values are used only if the app falls back to offline (REST) mode.

10. Deployment Guide

10.1 Server Setup

```
# 1. Create and activate virtualenv
python -m venv .venv
.venv\Scripts\activate          # Windows
source .venv/bin/activate          # Linux / macOS

# 2. Install dependencies
cd server
pip install -r requirements.txt

# 3. Configure environment
copy .env.example .env             # then edit .env with keys

# 4. Ensure storage root exists  (or set FI_STORAGE_ROOT)
mkdir D:\fig

# 5. Start server
python main.py
#   or for production:
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

10.2 Android Build

- Open android/ in Android Studio (Hedgehog 2023.1.1 or later)
- Update serverUrl in FiConfig.kt to the server's LAN IP address
- Build → Generate Signed APK (or Run on connected device in debug)
- Minimum SDK: API 26 (Android 8.0); Target SDK: 34
- Ensure the Android device and server are on the same LAN / VPN
- Grant all permissions when prompted on first launch

10.3 Testing Checklist

1	GET /health	200 OK with config summary
2	WS connect to /ws/fi-session/test123	Connection accepted; server waits for {ready}
3	Send {ready}	Server sends {question, tts_audio} for Q1
4	Send {tts_done}	Server sends {start_listening}
5	Stream 5 s of PCM then {audio_end}	Server sends {transcript, is_final:true}
6	Complete all questions	Server advances to photo phase
7	Send {photo} with base64 JPEG + GPS	Server saves file; sends {photo_ack}
8	Send all photos then session completes	Server sends {session_done}
9	Wait 30–60 s after session	PDF appears in server/reports//
10	Open PDF	Customer details, geo table, image analysis visible
11	Geo outlier test (fake GPS far away)	Geo section shows FAILED with outlier name
12	Polly disabled (wrong key)	Session continues; audio not played

13	OpenAI disabled (empty key)	Report generated with '■ Analysis skipped' text

---

For issues or contributions, review the log output at INFO level. All server log lines are prefixed with the originating component in brackets: [Polly], [Transcribe], [Conductor], [Geo], [OpenAI], [Report]. Android logs are prefixed FiAgent/ and visible in Android Studio Logcat.